

GRANITI VICENTIA

V1 Agentic Shop Drawing Review Architecture

Final technology stack, workflow, safety boundaries, retrieval design, implementation plan and cost posture

Research verified: 30 July 2026

Core operating principle

The AI reads. The math decides. A human signs off.

Primary safety metric

Critical false-PASS rate. A wrong FAIL creates review work; a wrong PASS can be manufactured.

Architecture status: V1 approved target

1. Executive decision

Final V1 selection

Hatchet OSS orchestrates the package workflow. One bounded LangGraph agent handles ambiguous extraction. A canonical evidence model and validation gate protect an isolated deterministic Python verdict engine. PostgreSQL is the authoritative record; hybrid retrieval is advisory only.

Decision area	V1 choice	Why
Outer workflow	Hatchet OSS / Hatchet Lite	Low fixed cost, MIT licence, PostgreSQL-backed, task history, retries, parallel DAGs and durable event waits.
Agentic layer	Bounded LangGraph extraction agent	Dynamic tool routing is useful only when a page is ambiguous; the graph is capped and cannot issue verdicts.
Verdict	Isolated typed Python engine	Exact arithmetic, reproducible results and no model, memory or retrieval access.
Data plane	PostgreSQL + S3	One relational source of truth plus immutable source/evidence artifacts.
Retrieval	Exact ID + pg_trgm + BM25 + dense	Protects coded identifiers while preserving semantic recall; all outputs remain advisory.
Production upgrade	Temporal only after measured triggers	Do not pay production workflow overhead before the extraction core is proven.

This is a workflow-first agentic system: the workflow is durable, the agent is bounded, the verdict is non-agentic, and the reviewer retains final authority.

2. V1 product boundary

In V1	Deferred
Cabinets and countertops only	Lighting, seating and other categories
Selected dimensions, materials and item identifiers	Full drawing-domain automation
Digitally generated PDFs as primary path	Scanned/photographed drawings as a primary promise
Basic scanned-page fallback	Custom detector training
One GV team and a small reviewer group	General multi-tenant enterprise platform
Manual YAML rule management	Rule-authoring product
Evidence-linked findings and redlined PDF	Automatic vendor submission
Reviewer corrections captured	Automatic rule learning

3. Final V1 technology stack

Use now versus defer is explicit to keep the architecture lean.

Layer	Choice	V1 role
Language	Python 3.12+	Core implementation
API	FastAPI + Pydantic	Typed APIs and data contracts
Frontend	React/Vite + TypeScript	Lean reviewer application
PDF viewer	PDF.js	Side-by-side evidence review
Workflow	Hatchet OSS / Hatchet Lite	Jobs, retries, DAGs, parallel page work
Agentic extraction	LangGraph	Bounded decision loop only
Database	PostgreSQL	Authoritative records and Hatchet state
Files	Amazon S3	Source PDFs, crops, redlines and reports
PDF repair	pikepdf	Normalize malformed PDFs
PDF rendering	pypdfium2	Stable page images and crops
Vector extraction	pdfplumber	Characters, words, lines, rectangles and curves
Primary OCR	PaddleOCR	Raster and difficult-page text
OCR verifier	docTR	Critical or conflicting values only
Image processing	OpenCV	Rotation, skew and enhancement
Geometry	Shapely + custom Python	Spatial relationships and evidence polygons
VLM	Amazon Nova 2 Lite via Bedrock	Ambiguous cropped regions only
Exact/fuzzy IDs	B-tree aliases + pg_trgm	Protect coded identifiers
Sparse retrieval	bm25s	Low-infrastructure lexical retrieval
Dense retrieval	pgvector + BGE-small-en-v1.5	Advisory semantic retrieval
Fusion	Reciprocal Rank Fusion	Combine sparse and dense rankings
Rulebook	YAML -> validated JSON	Reviewable and versionable rules
Verdict	Typed Python operation registry	No eval; exact, testable operations
Arithmetic	Decimal + Fraction	Exact units and fractions
Outputs	ReportLab + pypdf + openpyxl	PDF redlines and spreadsheet reports
Testing	pytest gold-set harness	False-PASS and evidence accuracy measurement
Deployment	Docker Compose on one 8 GB VM	Low operational cost

4. System architecture

The agentic layer surrounds the deterministic core; it never replaces it.

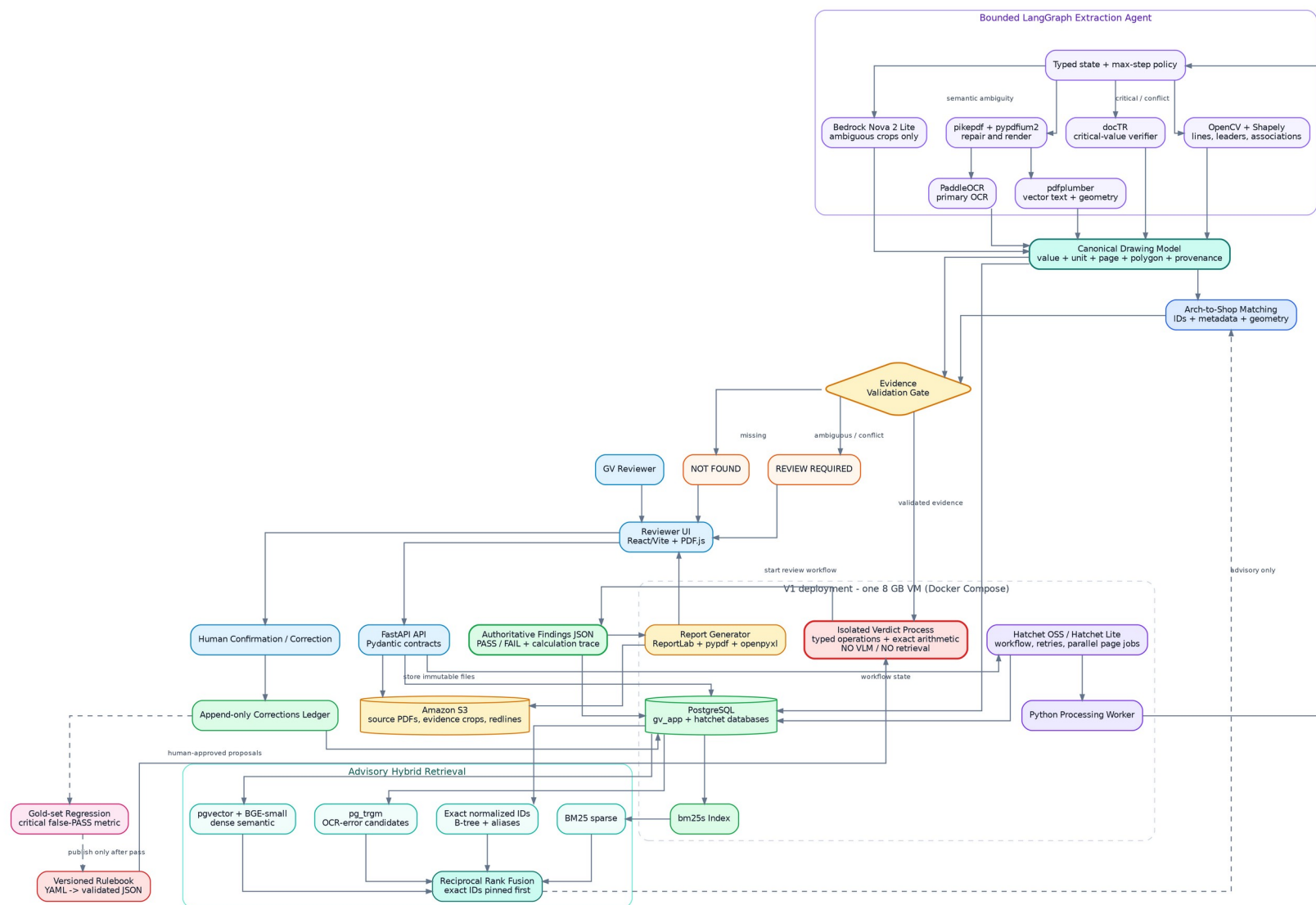


Figure 1. Final V1 system architecture. Dashed retrieval and correction paths are advisory or governance paths, not verdict authority.

5. End-to-end processing flow

- 1.The reviewer uploads the architectural set and vendor shop drawing.
- 2.The API hashes the files, records immutable document versions and stores originals in S3.
- 3.Hatchet creates a package workflow and schedules page classification and extraction jobs.
- 4.Each page is routed through vector extraction, OCR, geometry analysis and, only when required, a cropped VLM interpretation step.
- 5.All candidates are converted into typed canonical observations with source page, polygon, raw text, normalized value, unit and extractor versions.
- 6.Arch-to-shop matching uses authoritative identifiers and geometry, supported by advisory exact, sparse and dense retrieval.
- 7.The Evidence Validation Gate blocks missing, conflicting, ambiguous or non-authoritative inputs.
- 8.The isolated Python engine executes a published typed rule with exact arithmetic.
- 9.A versioned finding is emitted as PASS, FAIL, NOT FOUND or REVIEW REQUIRED, with the calculation and evidence.
10. The reviewer confirms or corrects the finding; corrections enter an append-only ledger and never become rules automatically.

Outcome	Definition	Authority
PASS	Validated values satisfy the published rule and tolerance.	Deterministic engine
FAIL	Validated values violate the published rule or tolerance.	Deterministic engine
NOT FOUND	A required authoritative input is absent.	Evidence gate
REVIEW REQUIRED	Evidence conflicts, semantics are ambiguous, or human judgment is required.	Evidence gate / reviewer

6. Component responsibility boundaries

Component	Owns	Must not own
Hatchet OSS	Package state, task scheduling, retries, page parallelism, status, cancellation and durable waits.	OCR interpretation or PASS/FAIL.
LangGraph agent	Dynamic extraction tool selection, bounded retries, crop refinement and ambiguity escalation.	Rule selection, tolerance calculation or package approval.
Canonical model	Typed facts, provenance, coordinates, evidence state and authority class.	Implicit assumptions or hidden prompt state.
Hybrid retrieval	Candidate discovery, historical context and suggested matches.	Authoritative dimensions or exceptions.
Evidence gate	Eligibility of observations and matches for deterministic checking.	Inventing missing values.
Verdict engine	Unit normalization, exact calculations and typed rule execution.	Models, retrieval, memory, internet access or arbitrary code.
Reviewer	Final approval, correction and documented exceptions.	Silent untracked changes.

7. Bounded agentic extraction

Where the system is agentic

The extraction graph observes the current evidence and chooses the next permitted tool. It can retry, crop and escalate, but its only safe terminal states are a corroborated observation or an explicit abstention.

```
Inspect page
-> clean vector evidence?
  yes -> pdfplumber + rendered verification
  no -> PaddleOCR
-> critical or conflicting value?
  yes -> doctR cross-check
-> semantic association unclear?
  yes -> geometry inspection + cropped Nova call
-> evidence corroborated?
  yes -> emit typed observation
  no -> REVIEW REQUIRED
```

Guardrail	V1 policy
Maximum graph steps	6-8
OCR retries	Maximum 2
VLM calls per ambiguous region	1 primary call + 1 escalation at most
VLM input	Targeted crop plus bounded context; not the full package by default
Outputs	Pydantic schema only
Numeric disagreement	Never resolved by model preference; escalate
Rule modification	Prohibited
Verdict authority	Prohibited
Unresolved state	REVIEW REQUIRED

LangGraph provides checkpointed state and interrupts for human-in-the-loop flows, but the V1 uses it only inside the extraction activity rather than as the business-process backbone. [R5-R7]

8. Canonical Drawing Model and evidence gate

Observation field	Purpose
document_version_id + hash	Pins the fact to an immutable source revision.
document_role	ARCH or SHOP authority context.
page_number + source_polygon	Exact evidence location.
evidence_crop_uri	Reviewer-verifiable visual proof.
raw_text	Preserves what the extractor actually read.
normalized_value + canonical_unit	Provides deterministic operands.

Observation field	Purpose
semantic_type	Example: sink_cutout_width, cabinet_depth, material.
item/view relationships	Connects values to products and drawing views.
extraction_methods + versions	Supports reproducibility and regression analysis.
evidence_status	RAW_CANDIDATE, CORROBORATED, CONFLICTING, HUMAN_CONFIRMED or REJECTED.
authority_class	AUTHORITATIVE or ADVISORY.
ambiguity_flags	Records unresolved or judgmental conditions.

Evidence status	May enter verdict?	Meaning
RAW_CANDIDATE	No	Only one unverified extraction route produced it.
CORROBORATED	Yes	Independent evidence agrees and the semantic association is valid.
HUMAN_CONFIRMED	Yes	Reviewer explicitly confirmed the value and evidence.
CONFLICTING	No	Readers or associations disagree.
REJECTED	No	Candidate was found invalid.

Automatic PASS eligibility

A check type is not eligible for automatic PASS until its source, numeric value, unit, item match and evidence polygon meet the release threshold on the held-out gold set.

9. Advisory hybrid retrieval

Dense embeddings alone are unsafe for coded identifiers such as X-223 versus X-233. V1 therefore uses an exact-first retrieval policy, then lexical and semantic retrieval for recall.

Priority	Lane	Purpose
1	Exact normalized identifier	Pin the correct code before ranking.
2	Alias table	Resolve known forms such as X223, X-223 and X 223.
3	Hard metadata filters	Restrict project, revision, product category and document role.
4	pg_trgm candidates	Surface possible OCR mistakes without silently accepting them.
5	BM25 sparse retrieval	Match rare codes, drawing terminology and exact phrases.
6	Dense pgvector retrieval	Find semantic equivalents and related descriptions.
7	Reciprocal Rank Fusion	Fuse ranked lists; exact ID remains pinned.

Raw code	Normalized code
X-223	X223
x 223	X223
X-223	X223
VAN-01-A	VAN01A

PostgreSQL pg_trgm provides indexed alphanumeric similarity; pgvector explicitly supports hybrid use with lexical search and mentions Reciprocal Rank Fusion; bm25s supplies a low-dependency in-process BM25 implementation. [R13-R16]

Retrieval trust boundary

A retrieved result may suggest where to look or which items may correspond. It may not supply an approved measurement, create an exception, override a source document or directly feed the verdict engine.

10. Deterministic rulebook and verdict engine

Rule design	V1 decision
Authoring	YAML for human review
Published representation	Validated canonical JSON snapshot
Validation	Pydantic + JSON Schema
Execution	Small typed operation registry in Python
Arithmetic	Fraction for architectural fractions; Decimal for canonical values

Rule design	V1 decision
Canonical unit	Millimetres unless a rule explicitly requires another unit
Arbitrary eval / executable formulas	Prohibited
Versioning	Every finding stores the exact rule snapshot ID

```
id: CT-SINK-004
version: 1.0.0
product_type: countertop
expected:
  source: ARCH
  semantic_type: sink_cutout_width
actual:
  source: SHOP
  semantic_type: sink_cutout_width
operation:
  type: within_tolerance
  tolerance: { value: 3, unit: mm }
on_missing: NOT_FOUND
on_ambiguous: REVIEW_REQUIRED
```

Initial typed operations
exists
equals
within_tolerance
minimum / maximum
between
one_of
contains
count_equals
conditional_required
difference_between

Verdict service isolation
The verdict process has no VLM credentials, no vector or BM25 access, no historical corrections, no conversational memory, no arbitrary formula execution and no general outbound internet access.

11. Extraction stack rationale and licensing

Tool	Why selected	Boundary
pdfplumber	MIT licensed; exposes characters, words and vector objects; suitable for machine-generated PDFs.	Not a scanner OCR engine. [R8-R9]
pypdfium2	Liberal licensing and stable PDFium-based rendering.	Rendering and visual verification, not semantic interpretation. [R10]
pikepdf	MPL-2.0; robust repair and normalization through qpdf.	Source-level modifications to pikepdf itself require MPL compliance. [R11]
PaddleOCR	Apache-2.0; active OCR/document parsing ecosystem.	Must be benchmarked on GV drawings rather than accepted from generic benchmarks. [R12]
docTR	Apache-2.0 independent detection/recognition implementation.	Use selectively for critical or conflicting values. [R13]
Nova 2 Lite	Cost-efficient multimodal model designed for document processing.	Cropped semantic interpretation only; never a verdict source. [R17]

Tool not selected	Reason
PyMuPDF	Dual AGPL/commercial licence creates a hosted proprietary product decision that must be separately approved. [R19]
Ultralytics YOLO	Official guidance requires AGPL disclosure or an Enterprise licence for private/commercial use. [R20-R21]
Generic trained detector in V1	Adds annotation, training and inference complexity before geometry-first methods have been benchmarked.
Full-page VLM as primary reader	Higher cost, weaker evidence localization and greater hallucination risk than vector/OCR-first extraction.

Amazon Bedrock model providers do not have access to Bedrock customer prompts, completions or service logs, which supports the privacy posture for proprietary drawings. [R18]

12. Lean deployment and operating cost

```

One 8 GB VM (Docker Compose)
- reverse proxy
- React/Vite frontend
- FastAPI API
- Hatchet Lite
- Hatchet Python worker
- PostgreSQL: gv_app database + hatchet database
- isolated verdict process
External
- Amazon S3
- Amazon Bedrock

```

Cost item	Planning basis	Monthly planning estimate
4 GB Lightsail option	US\$24 public-IPv4 Linux bundle; suitable for experiments but tight with OCR.	~Rs 2,300 before tax and variable services
8 GB Lightsail recommendation	US\$44 public-IPv4 Linux bundle.	~Rs 4,230 before tax and variable services
S3	Pay-as-used storage and requests; no minimum charge.	~Rs 200-600 at V1 volume
Bedrock VLM	Only ambiguous crops reach the model.	~Rs 500-2,000 depending on package difficulty
Domain, backups and logs	Lean retention and daily database backup to S3.	~Rs 300-900
Hatchet OSS, LangGraph and open-source tools	No software licence fee.	Rs 0 software fee

Recommended V1 operating budget

Rs 6,000-9,000 per month before taxes, engineering labour and unusual model usage. Start locally during the gold-set phase; deploy the 8 GB VM only when reviewer access is needed.

The Lightsail price is current list pricing; the rupee conversion uses the RBI/FBIL reference rate of Rs 96.1856 per US dollar from 27 July 2026. Actual billing varies by region, exchange rate, taxes and usage. [R22-R25]

13. Implementation plan - build in risk order

Phase	Build	Exit gate
0. Gold set	Select and annotate representative cabinet and countertop packages.	Ground truth contains values, units, identifiers, polygons, matches and expected findings.
1. Small core loop	Local upload -> extraction -> evidence -> deterministic check.	Exact values and evidence locations are measurable.
2. Canonical model	Formalize typed observations and coordinate transforms.	No raw model output can reach checking.
3. Evidence gate + rules	Implement eligibility policy and typed operations.	Unsupported inputs become NOT FOUND or REVIEW REQUIRED.
4. Agentic routing	Add LangGraph only around ambiguous page handling.	It must improve accuracy, cost or review time versus fixed routing.
5. Hybrid retrieval	Exact identifiers, aliases, pg_trgm, BM25, dense and RRF.	Match precision improves without increasing false PASS.
6. Hatchet wrapper	Add durable workflow, parallel page tasks and retries.	Measured core runs reliably as jobs.
7. Reviewer UI + outputs	Evidence-linked findings, redlines and corrections.	Reviewer can validate every decision quickly.

Ordering principle

Do not build the durable platform before proving extraction accuracy on real GV drawings. The architecture is the target; the implementation sequence retires the riskiest unknown first.

14. Evaluation and release gates

Priority	Metric	Why
1	Critical false-PASS rate	Manufacturing risk and primary release blocker.
2	Evidence page and polygon correctness	Every finding must be immediately verifiable.
3	Numeric and unit exact-match accuracy	Prevents silent arithmetic on bad inputs.
4	Identifier and item-match precision	Wrong matches contaminate otherwise correct checks.
5	FAIL recall	Ensures errors are found.
6	NOT FOUND and REVIEW REQUIRED recall	Measures honest abstention.
7	Reviewer correction rate	Shows where automation remains unreliable.

Priority	Metric	Why
8	Reviewer minutes per package	Measures actual product value.
9	Automation coverage	Optimized only after safety metrics.

Release gate	Required behaviour
OCR disagreement	Never auto-resolve; REVIEW REQUIRED.
Unknown unit	Cannot enter verdict.
Missing approved source	NOT FOUND.
Advisory retrieval result	Cannot become a verdict operand.
Rule change	Human approval plus full gold-set regression.
New automatic PASS check type	Separate acceptance decision based on held-out results.

15. Upgrade triggers and deferred production architecture

Trigger	Upgrade
Frequent workflow recovery problems or complex in-progress code migrations	Evaluate Temporal Cloud or self-hosted Temporal.
More than 5-10 concurrent packages or sustained worker contention	Separate API and processing workers.
Database downtime becomes commercially material	Move PostgreSQL to RDS and introduce managed backups.
Formal high availability requirement	Move deployment to ECS/Fargate plus Multi-AZ RDS.
Managed VLM spend is consistently higher than GPU economics	Benchmark self-hosted Qwen-family VLM through vLLM.
Embedded BM25 misses latency/scale target	Introduce a dedicated lexical search service such as OpenSearch.
pgvector materially affects transactional workload	Move vector search to Qdrant or another dedicated service.
Multi-hop relationship queries improve held-out matching	Evaluate a graph layer; do not add GraphRAG by default.
Several external systems need a common agent-controlled integration layer	Evaluate MCP.

Temporal remains the preferred production-grade workflow target when the process becomes mission-critical; current Temporal Cloud AWS Marketplace pricing lists a US\$100 monthly plan fee plus usage, which is intentionally avoided in V1. [R26-R27]

16. Key risks and controls

Risk	Control
Numerically plausible but incorrect extraction	Independent verification, evidence polygons and strict eligibility states.
Wrong item/view association	Exact identifiers first, metadata and geometry checks, human confirmation for ambiguity.
VLM hallucination	Crop-bounded structured outputs; VLM cannot create verdict authority.
Agent loops or cost runaway	Maximum steps, call budget and mandatory abstention.
False PASS	Primary metric, check-type release gates and no inference-based tolerance decisions.
Rule injection or unsafe expressions	Typed operation registry; no eval.
Retrieval contaminates truth	Authority class enforced; verdict process has no retrieval access.
Revision confusion	Immutable document hashes and revision-aware source selection.
Licensing surprise	Approved dependency list and model-weight licence review.
Corrections silently become rules	Append-only ledger, human proposal gate and full regression.

Final architecture statement

Hatchet orchestrates. LangGraph investigates. The Canonical Drawing Model preserves evidence. Hybrid retrieval advises. The Evidence Gate protects the boundary. Exact Python rules decide. The reviewer signs off.

Appendix A. Mermaid architecture definition

```

flowchart TB
    U[GV Reviewer] --> UI[Reviewer UI - React/Vite + PDF.js]
    UI --> API[FastAPI API]
    API --> S3[(Amazon S3)]
    API --> H[Hatchet OSS / Hatchet Lite]
    API --> PG[(PostgreSQL: gv_app)]

    subgraph V1[One 8 GB VM - Docker Compose]
        H --> W[Python Worker]
        H --> HDB[(PostgreSQL: hatchet)]
        W --> LG
        V[Isolated Deterministic Verdict Process]
        R[Report Generator]
    end

    subgraph LG[Bounded LangGraph Extraction Agent]
        P[pikpdf + pypdfium2]
        X[pdflumber]
        O1[PaddleOCR]
        O2[docTR verifier]
        G[OpenCV + Shapely]
        M[Nova 2 Lite - cropped ambiguity only]
        P --> X
        P --> O1
        O1 --> O2
        X --> G
        O1 --> G
        G --> M
    end

    X --> CDM[Canonical Drawing Model]
    O1 --> CDM
    O2 --> CDM
    G --> CDM
    M --> CDM
    CDM --> PG

    subgraph RET[Advisory Hybrid Retrieval]
        E[Exact IDs + aliases]
        F[pg_trgm]
        B[BM25S]
        D[pgvector + BGE-small]
        Q[Reciprocal Rank Fusion - exact pinned]
        E --> Q
        F --> Q
        B --> Q
        D --> Q
    end

    PG --> E
    PG --> F
    PG --> B
    PG --> D
    CDM --> MATCH[Arch-to-Shop Matching]
    Q -. advisory only .-> MATCH
    MATCH --> GATE[Evidence Validation Gate]
    CDM --> GATE
    GATE -->|missing| NF[NOT FOUND]
    GATE -->|ambiguous or conflicting| RR[REVIEW REQUIRED]
    GATE -->|validated evidence| V
    RULES[Versioned YAML -> validated JSON rules] --> V
    V --> FIND[Authoritative Findings JSON]
    FIND --> R
    R --> S3
    R --> UI
    UI --> HC[Human confirmation or correction]
    HC --> LEDGER[Append-only corrections ledger]
    LEDGER --> PG
    LEDGER -. approved rule proposal .-> GOLD[Gold-set regression]
    GOLD -. publish after pass .-> RULES

```

The rendered architecture in Figure 1 is the presentation version of this Mermaid definition. The Mermaid source is the portable architecture-as-code reference for engineering documentation.

Appendix B. Research references

Official documentation and primary project sources verified on 30 July 2026.

- [R1] Hatchet Lite Deployment - designed for development and low-volume use; PostgreSQL can be both database and message queue. <https://docs.hatchet.run/self-hosting/hatchet-lite>
- [R2] Hatchet Durable Tasks - checkpoints, event waits, child tasks and deterministic replay. <https://docs.hatchet.run/v1/durable-tasks>
- [R3] Hatchet DAGs as Durable Workflows - persisted task results, retries and automatic parallelism. <https://docs.hatchet.run/v1/directed-acyclic-graphs>
- [R4] Hatchet pricing and open-source position. <https://hatchet.run/pricing>
- [R5] LangGraph overview - stateful workflows and durable agent execution. <https://docs.langchain.com/oss/python/langgraph/overview>
- [R6] LangGraph persistence - checkpoints, fault tolerance and replay. <https://docs.langchain.com/oss/python/langgraph/persistence>
- [R7] LangGraph interrupts - pause and resume human-in-the-loop graphs. <https://docs.langchain.com/oss/python/langgraph/interrupts>
- [R8] pdfplumber project documentation - machine-generated PDF focus and geometry objects. <https://github.com/jsvine/pdfplumber>
- [R9] pdfplumber MIT licence. <https://github.com/jsvine/pdfplumber/blob/stable/LICENSE.txt>
- [R10] pypdfium2 project and licensing notes. <https://github.com/pypdfium2-team/pypdfium2>
- [R11] pikepdf project and MPL-2.0 licence. <https://github.com/pikepdf/pikepdf>
- [R12] PaddleOCR project - Apache-2.0 OCR and document parsing toolkit. <https://github.com/PaddlePaddle/PaddleOCR>
- [R13] docTR project - Apache-2.0 OCR detection and recognition. <https://github.com/mindee/doctr>
- [R14] PostgreSQL pg_trgm documentation - indexed alphanumeric similarity. <https://www.postgresql.org/docs/17/pgtrgm.html>
- [R15] pgvector project - vector search and hybrid-search guidance including RRF. <https://github.com/pgvector/pgvector>
- [R16] BM25S - low-dependency in-process BM25 implementation. <https://bm25s.github.io/>
- [R17] Amazon Nova 2 Lite model card - cost-efficient multimodal document processing. <https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-2-lite.html>
- [R18] Amazon Bedrock data protection - model providers cannot access prompts, completions or service logs. <https://docs.aws.amazon.com/bedrock/latest/userguide/data-protection.html>
- [R19] PyMuPDF package licence - AGPL or Artifex commercial licence. <https://pypi.org/project/pymupdf/>
- [R20] Ultralytics licensing guidance - AGPL or Enterprise for proprietary/commercial use. <https://www.ultralytics.com/license>
- [R21] Ultralytics repository AGPL-3.0 licence. <https://github.com/ultralytics/ultralytics/blob/main/LICENSE>
- [R22] Amazon Lightsail instance bundle pricing. <https://docs.aws.amazon.com/lightsail/latest/userguide/amazon-lightsail-bundles.html>
- [R23] Amazon S3 pricing - pay for storage, requests and transfer; no minimum charge. <https://aws.amazon.com/s3/pricing/>
- [R24] RBI/FBIL exchange reference used for planning conversion, 27 July 2026. https://m.rbi.org.in/scripts/BS_ForeignExchangeDisplay.aspx?prid=59058
- [R25] ReportLab open-source BSD licensing. <https://docs.reportlab.com/developerfaqs/>
- [R26] Temporal platform documentation - durable crash recovery and long-running workflows. <https://docs.temporal.io/>
- [R27] Temporal Cloud through AWS Marketplace - US\$100 monthly plan fee plus usage. <https://temporal.io/get-cloud/aws-marketplace>
- [R28] BAAI BGE-small-en-v1.5 model card - MIT licensed 33.4M-parameter embedding model. <https://huggingface.co/BAAI/bge-small-en-v1.5>